Learning Objectives

The primary objective is to practice the following workflow (provided that no errors occurred during compilation): **edit (and save) → compile → run**

Programs should be developed by starting with a small example and extending it step by step. In the final step—the outlook—the initial example is developed into a very simple compiler.

The worksheet also addresses the following secondary objectives:

- reviewing material from the lectures, as demonstrated in the videos,
- learning new language features through examples,
- gaining experience with the shell.

Step 1

The functions `getchar` and `putchar` are declared in `stdio.hdr`:

- `getchar` reads a character from the keyboard and returns it.
- `putchar` takes a character as an argument and prints it to the screen.

The program shown on the right, `ex1.abc`, repeatedly performs the following steps in an infinite loop: it reads a character, prints it, reads another character, prints it, and so on.

```
@ <stdio.hdr>

fn main()
{
    while (true) {
        putchar(getchar());
    }
}
```

Tasks

1. Type the program into your preferred editor. Compile and run it. Pay particular attention to the following: characters are apparently not printed immediately, but only after you press Return. For example, if you enter `a`, `b`, `c`, and then press `Return`, you obtain the following output:

   ```
   MCL:session1 lehn$ ./a.out
   abc
   abc
   ```

2. You can terminate the program forcibly with `Control-C`, since it runs in an infinite loop.
3. Our programs run inside the shell. Therefore, the statements above about `getchar` and `putchar` were not quite correct. A more accurate description—in the spirit of fuzzy logic—is:
   (A) `getchar` does not receive its input directly from the keyboard, but from the shell.
   (B) `putchar` does not write characters directly to the screen, but passes them to the shell for output.
   (C) By default, the shell passes the input to our program only after you have pressed `Return`. This allows you to correct your input—for example, using `Backspace`—before pressing `Return`.
   (D) You can use the shell command `stty` to change this behavior so that input is passed to the program immediately:

   ```
   MCL:session1 lehn$  stty -icanon min 1
   MCL:session1 lehn$  ./a.out
   aabbcc
   ```

   (E) The shell is now effectively “unusable”—after all, we normally want to be able to correct our input before pressing Return. You should therefore close the terminal and open a new one[1].
4. The keyword `true` is used as the condition in the program. Internally, the compiler replaces `true` with `1`. More generally, any integer value can be used as a condition. The rule is: `0` is `false`, and every other value is `true`.

   We will test this using the workflow **edit and save → compile → test**

   (A) Replace `true` with `42`.
   (B) Replace `true` with `237` > `42`.

[1] Note: The terminal settings can, in principle, be restored with `stty`. Since we have not found a platform-independent command that reliably restores the original settings on all systems used in this course, we recommend simply closing the terminal and opening a new one.

© Michael C. Lehn | CC BY-SA 4.0

Step 2

Characters are, in fact, just integer values that are interpreted as characters. For example, `'A'` has the value `65` and `'B'` has the value `66`. You can look up this encoding in an ASCII table. Alternatively, you can write a program that prints the corresponding integer value.

Tasks

1. Modify the program as follows:

```
@ <stdio.hdr>

fn main()
{
    while (true) {
        local ch: int = getchar();
        printf("ch = %d\n", ch);
    }
}
```

2. Compile and run the program. If you now enter `a`, `b`, `c`, and then press `Return`, you should obtain the following output:

```
MCL:session1 lehn$  ./a.out
abc
ch = 97
ch = 98
ch = 99
ch = 10
```

   This tells us that the ASCII value of `'a'` is `97`, the ASCII value of `'b'` is `98`, the ASCII value of `'c'` is `99`, and the ASCII value of `Return` is `10`.
   Determine the ASCII values of the digits `'0'`, `'1'`, …, `'9'`.
3. It is inconvenient that the program always has to be terminated with `Control-C`. Modify it as follows:

```
@ <stdio.hdr>

fn main()
{
    while (true) {
        local ch: int = getchar();
        if (ch == '.') {
            break;
        } else {
            printf("ch = %d\n", ch);
        }
    }
}
```

   Run the program again using an input that contains a `'.'`, for example `a`, `b`, `c`, `.`. The program should now terminate as soon as the character `'.'` is read.
4. It would be even better if the program terminated automatically when no more input is available. Modify the `if` condition as follows:

```
if (ch == EOF) {
```

   You can now terminate the program by pressing `Control-D`. This may not seem like much of an improvement over `Control-C`, but it will soon become clear why it is.
5. `EOF` stands for *End of File*. Try the following command (if your ABC source file is not called `ex1.abc`, replace the filename accordingly):

```
MCL:session1 lehn$ ./a.out < ex1.abc
ch = 64
ch = 32
…
```

   We will learn more about the shell later in the course. For now, simply observe that, in this example, the program reads its input from the file `ex1.abc` rather than from the keyboard.

© Michael C. Lehn | CC BY-SA 4.0

6. Internally, the compiler replaces `EOF` with an integer value. Although this value is platform-dependent, it is usually `-1`. Verify that the program behaves exactly the same when `EOF` is replaced with `-1`. Afterwards, restore `EOF`—it makes the program much easier to read.
7. Modify the format string and the `printf` statement as follows:

```
@ <stdio.hdr>

fn main()
{
    while (true) {
        local ch: int = getchar();
        if (ch == EOF) {
            break;
        } else {
            printf("ch = %d -> '%c'\n", ch, ch);
        }
    }
}
```

   The program now prints both the ASCII value (using the `%d` format specifier) and the corresponding character (using the `%c` format specifier).
8. Modify the program as follows:

```
@ <stdio.hdr>

fn main()
{
    while (true) {
        local ch: int = getchar();
        if (ch >= '0' && ch <= '9') {
            printf("digit '%c'\n", ch);
        } else if (ch == EOF) {
            break;
        } else {
            printf("ch = %d -> '%c'\n", ch, ch);
        }
    }
}
```

   The program now treats the digits `'0'` through `'9'` as a special case. Test it with an input such as `a1cd4`. You should obtain the following output:

```
MCL:session1 lehn$ ./a.out
a1cd4
ch = 97 -> 'a'
digit '1'
ch = 99 -> 'c'
ch = 100 -> 'd'
digit '4'
ch = 10 -> '
'
```

   Notice the newline character at the end of the output.
9. In addition to the ASCII value of a digit, the program should now also print its numeric value. By numeric value we mean the following:
   - The character `'0'` has the ASCII value `48`, but the program should print `0`.
   - The character `'1'` has the ASCII value `49`, but the program should print `1`.
   - …
   - The character `'9'` has the ASCII value `57`, but the program should print `9`.

   This can be achieved by subtracting the ASCII value of `'0'` from the ASCII value of the digit. Modify the `printf` statement as follows:

```
printf("digit '%c', value = %d\n", ch, ch - '0');
```

   Run the program again with an input such as `a1cd4`.

© Michael C. Lehn | CC BY-SA 4.0

Step 3

Things are now getting a little more complex. Instead of recognizing individual digits, the program should now recognize complete integer values in the input. You are given a template for this. First test the program; afterward, we will examine the details more closely.

Modify or extend your program as follows:

```
@ <stdio.hdr>

fn main()
{
    while (true) {
        local ch: int = getchar();
        if (ch >= '0' && ch <= '9') {
            local val: int = 0;
            while (ch >= '0' && ch <= '9') {
                val = val * 10;
                val = val + ch - '0';
                ch = getchar();
            }
            printf("integer with value %d\n", val);
        } else if (ch == EOF) {
            break;
        } else {
            printf("ch = %d -> '%c'\n", ch, ch);
        }
    }
}
```

Tasks

1. Test the program with an input such as `abc123xy42`. It should correctly recognize that the integer values `123` and `42` occur in the input:

   ```
   MCL:session1 lehn$ ./a.out
   ab123xy42
   ch = 97 -> 'a'
   ch = 98 -> 'b'
   integer with value 123
   ch = 121 -> 'y'
   integer with value 42
   ```

   However, the program does not yet work correctly: the character `'x'` has been “swallowed.” More generally, the character immediately following each integer is swallowed. We will fix this later.
2. Now examine the part of the program that recognizes integers in the input and ensures that the variable `val` contains the corresponding numeric value afterward:

   ```
   local val: int = 0;
   while (ch >= '0' && ch <= '9') {
       val = val * 10;
       val = val + ch - '0';
       ch = getchar();
   }
   ```

   Explain to the student sitting next to you—or ask us to explain—why, for the input sequence `'1'`, `'2'`, `'3'`, the variable `val`
   (A) has the value `1` after the first iteration of the loop,
   (B) has the value `12` after the second iteration,
   (C) has the value `123` after the third iteration.
3. Use the operators `*=` and `+=` inside the while loop.
4. Explain to the student sitting next to you why one character is swallowed after each integer. You will see how to solve this problem in the next step.

© Michael C. Lehn | CC BY-SA 4.0

Step 4

The problem of characters being swallowed after a sequence of digits is fixed by the following modification:

```
 1 @ <stdio.hdr>
 2
 3 fn main()
 4 {
 5     local ch: int = getchar();
 6     while (true) {
 7         if (ch >= '0' && ch <= '9') {
 8             local val: int = 0;
 9             while (ch >= '0' && ch <= '9') {
10                 val *= 10;
11                 val += ch - '0';
12                 ch = getchar();
13             }
14             printf("integer with value %d\n", val);
15         } else if (ch == EOF) {
16             break;
17         } else {
18             printf("ch = %d -> '%c'\n", ch, ch);
19             ch = getchar();
20         }
21     }
22 }
```

Tasks

1. Verify that no character is swallowed anymore.
2. There are two important changes in the program:
   (A) The variable `ch` is now declared before the while loop. Its scope is therefore the entire function. The statements that assign a new value to `ch` are in lines 12 and 19.
   (B) In the `else` branch of the `if` statement, `getchar` is now also called and its return value is assigned to `ch` in line 19. Test why this is necessary by commenting out that line, that is, by changing it to

   ```
   // ch = getchar();
   ```

   Run the program again. Explain to the student sitting next to you what goes wrong and why. Then undo the change.
3. The program should now also handle the character `'+'` as a special case. Insert the following code after line 14:

   ```
   } else if (ch == '+') {
       printf("PLUS\n");
       ch = getchar();
   ```

   Test the program with an input such as `12 + 4`.
4. Modify the program so that
   (A) `'*'` produces the output `ASTERISK`,
   (B) `'('` produces the output `LPAREN` for “left parenthesis,”
   (C) `')'` produces the output `RPAREN` for “right parenthesis,”
   (D) `';'` produces the output `SEMICOLON`.

© Michael C. Lehn | CC BY-SA 4.0

Step 5

We now make a larger change. Don't worry too much about understanding every detail yet—part of this step is simply to practice editing a larger piece of code.

1. The code inside the `while` loop is moved into a separate function called `getToken`.
2. All variables are now global. In addition, there is a new variable called `token`. Whenever something interesting is recognized in the input, this variable is assigned a corresponding value—for example, `1` for an integer, `2` for a plus sign, and so on.
3. In `main`, the function `getToken` is called repeatedly and the value of `token` is printed. The loop terminates when token has the value `7`, since this means that `getToken` has encountered `EOF`.

```
1 @ <stdio.hdr>
2
3 global token: int = 0;
4 global ch: int = 0;
5 global val: int = 0;
6
7 fn getToken()
8 {
9     if (ch >= '0' && ch <= '9') {
10        val = 0;
11        while (ch >= '0' && ch <= '9') {
12            val = val * 10;
13            val = val + ch - '0';
14            ch = getchar();
15        }
16        printf("integer with value %d\n", val);
17        token = 1;
18    } else if (ch == '+') {
19        printf("PLUS\n");
20        ch = getchar();
21        token = 2;
22    } else if (ch == '*') {
23        printf("ASTERISK\n");
24        ch = getchar();
25        token = 3;
26    } else if (ch == ';') {
27        printf("SEMICOLON\n");
28        ch = getchar();
29        token = 4;
30    } else if (ch == '(') {
31        printf("LPAREN\n");
32        ch = getchar();
33        token = 5;
34    } else if (ch == ')') {
35        printf("RPAREN\n");
36        ch = getchar();
37        token = 6;
38    } else if (ch == EOF) {
39        token = 7;
40    } else {
41        printf("ch = %d -> '%c'\n", ch, ch);
42        ch = getchar();
43        token = 8;
44    }
45 }
46
47 fn main()
48 {
49    while (true) {
50        getToken();
51        if (token == 7) {
52            break;
53        } else if (token == 1) {
54            printf("token = %d, val = %d\n", token, val);
55        } else {
56            printf("token = %d\n", token);
57        }
58    }
59 }
```

© Michael C. Lehn | CC BY-SA 4.0

Tasks

1. For the moment, do not worry about whether this change is a good idea or not. Right now, the goal is simply to practice your copy-and-paste skills and become more familiar with your editor.

2. Remove all `printf` statements from `getToken`.

Step 6

An `enum` declaration can be used to introduce symbolic names for integer values. The compiler then replaces these names with the corresponding integer values. We have already seen one example: the compiler replaces `EOF` with `-1`. Such symbolic constants make the code easier to read.

In our example, the following enum declaration is useful:

```
enum TokenKind {
    EOI, // = 0 (end of input)
    BAD,
    INTEGER,
    PLUS,
    ASTERISK,
    SEMICOLON,
    LPAREN,
    RPAREN,
};
```

The symbol `EOI`, short for “end of input,” is assigned the value `0`, `BAD` is assigned `1`, `INTEGER` is assigned `2`, and so on. Starting with zero, the compiler assigns consecutive integer values to the symbols in the order in which they appear.

Tasks

1. Modify the program accordingly and test it.
2. If `getToken` does not recognize an integer, `'+'`, `'*'`, `'('`, `')'`, or `';'`, it currently assigns `BAD` to the variable token. Instead, an “invalid” character should be ignored, and `getToken` should continue searching for the next “interesting” part of the input.
   To achieve this, replace line 47, which currently assigns `BAD` to `token`, with

   ```
   47          getToken();
   ```

   As a result, spaces and newline characters are now ignored, as the following test demonstrates:

   ```
   MCL:session1 lehn$ ./
   a.out
   12 + 3 * (4 + 5);
   token = 2, val = 12
   token = 3
   token = 2, val = 3
   token = 4
   token = 6
   token = 2, val = 4
   token = 3
   token = 2, val = 5
   token = 7
   token = 5
   ```

© Michael C. Lehn | CC BY-SA 4.0

Outlook

What we have implemented in `getToken` is called a **lexer**. A lexer groups individual characters into so-called **tokens**. In our implementation, we have effectively built a **finite-state machine**.

A natural next step could be to use the lexer to implement a calculator that evaluates arithmetic expressions involving `+`, `*`, and parentheses.

Alternatively, the calculator could generate assembly code describing how the expression can be evaluated using assembly instructions.

Adding the code shown on the right after the function `getToken` does exactly that.

As an additional exercise in using your editor and improving your typing skills, type this code yourself and add it to your program.

Compile and run the program. Experiment with various arithmetic expressions consisting of integers, the operators `+` and `*`, and parentheses. Remember that each expression must be terminated with a semicolon (`;`). For example, the input

```
3 * (4 + 5);
```

should produce the following output:

```
        load 3, %1
        load 4, %2
        load 5, %3
        add %3, %2, %4
        mul %4, %1, %5
```

A program like this can already be called a compiler. A good compiler would, of course, evaluate the expression at compile time in this case and generate code such as

```
MCL:session1 lehn$ ./a.out
3 * (4 + 5);
        load 27, %1
```

This optimization is called constant folding.

```
 51 global reg: int;
 52
 53 fn parseSum(): int;
 54
 55 fn parseInt(): int
 56 {
 57     if (token == INTEGER) {
 58         getToken();
 59         printf("\tload %d, %%%d\n", val, ++reg);
 60         return reg;
 61     } if (token == LPAREN) {
 62         getToken();
 63         local sum: int = parseSum();
 64         if (sum < 0) {
 65             printf("sum expected\n");
 66             return -1;
 67         }
 68         if (token != RPAREN) {
 69             printf("expected ')'\n");
 70             return -1;
 71         }
 72         getToken();
 73         return sum;
 74     } else {
 75         return -1;
 76     }
 77 }
 78
 79 fn parseProd(): int
 80 {
 81     local prod: int = parseInt();
 82     if (prod < 0) {
 83         return -1;
 84     }
 85     while (token == ASTERISK) {
 86         getToken();
 87         local factor: int = parseInt();
 88         if (factor < 0) {
 89             printf("expected factor\n");
 90             return -1;
 91         }
 92         printf("\tmul %%%d, %%%d, %%%d\n", factor, prod, ++reg);
 93         prod = reg;
 94     }
 95     return reg;
 96 }
 97
 98 fn parseSum(): int
 99 {
100     local sum: int = parseProd();
101     if (sum < 0) {
102         return -1;
103     }
104     while (token == PLUS) {
105         getToken();
106         local summand: int = parseProd();
107         if (summand < 0) {
108             printf("expected summand\n");
109             return -1;
110         }
111         printf("\tadd %%%d, %%%d, %%%d\n", summand, sum, ++reg);
112         sum = reg;
113     }
114     return reg;
115 }
116
117 fn main()
118 {
119     getToken();
120     for (local sum: int = -1; (sum = parseSum()) >= 0; ) {
121         if (token == SEMICOLON) {
122             getToken();
123         }
124     }
125 }
```

© Michael C. Lehn | CC BY-SA 4.0